

Thesis amendments: resolved drafts and remaining items

This document works through the code-dependent and substantive writing corrections from the examiner report. For each item it provides draft text (prose and/or LaTeX) that can be copied into the amended thesis, using the numbers produced by the amendment experiments (multi-seed URBAN-SED study, DCASE few-shot study with random baseline, FSD50K reproduction runs of ATGNN/LHGNN, parameter/walltime benchmarks, and the FSD50K label co-occurrence analysis). Grammatical/style corrections and amendments that still require new experiments or decisions are listed in the final section as remaining items.

All new numbers are reproducible from the released repositories: github.com/shubhrsingh22/hypernet-sed, github.com/shubhrsingh22/dccase-fewshot-hypernet, github.com/shubhrsingh22/Local-Higher-GNN, github.com/shubhrsingh22/ATGNN (private at the time of writing; to be made public or archived before submission).

Chapter 1 - Introduction

1a. Code and data availability paragraph (new, for Chapter 1 or an appendix)

```
\section*{Code and Data Availability}
The experimental work in this thesis is supported by publicly released code. The
hypernetwork experiments of Chapter~3 are available at
\url{https://github.com/shubhrsingh22/hypernet-sed} (URBAN-SED) and
\url{https://github.com/shubhrsingh22/dccase-fewshot-hypernet} (few-shot bioacoustics);
the audio tagging models of Chapter~4 at
\url{https://github.com/shubhrsingh22/Local-Higher-GNN} and
\url{https://github.com/shubhrsingh22/ATGNN}; and the Chapter~5 systems at the
repositories referenced in Sections~5.2 and~5.3. Each repository contains the model
code, configuration files, training and evaluation scripts, environment
specifications, and dataset preparation instructions. Datasets are not
redistributed; official download links and preparation scripts are provided instead.
Results in this thesis correspond to tagged releases of these repositories
(release tags and commit hashes are given per chapter).
```

Insert the release tags/commit hashes once the repositories are tagged (remaining item R6).

1b. Measurable hypothesis (replacement for the hypothesis paragraph in 1.1.1)

The central hypothesis of this thesis is that graph neural networks, operating on nodes derived from time--frequency regions or learned embeddings, can match or exceed the task performance of convolutional and attention-based architectures of comparable or larger parameter count on standard machine listening benchmarks, while providing complementary advantages: (i) competitive accuracy without external (ImageNet) pretraining, (ii) parameter efficiency relative to Transformer baselines, and (iii) robustness of learned representations to signal degradations such as additive noise and reverberation. Improvement is assessed against published CNN and Transformer baselines (PANNs, PSLA, AST and task-specific baselines) under matched training pipelines, datasets and evaluation metrics (mAP for tagging, segment-based F1 for SED, top-1 hit rate for fingerprinting, retrieval metrics for similarity).

1c. Strengthened graph sparsification contribution (addition to Contributions)

A recurring design element across the proposed models is `\emph{learned graph sparsification}`: rather than operating on fully connected graphs, each architecture restricts message passing to a small, data-dependent neighbourhood (k -nearest neighbours in feature space in Chapters~4 and~5, top- n thresholded learned affinities in Chapter~5). This bounds the computational cost of graph construction, acts as a structural prior that suppresses spurious long-range connections, and mitigates the over-smoothing associated with dense aggregation (Section~2.3.7).

1d. Audio-as-graph visualisation

A figure is required (remaining item R8). Suggested composition: left panel, a log-mel spectrogram of a

polyphonic clip; middle panel, the same spectrogram overlaid with patch nodes and k-NN edges connecting harmonically/temporally related regions; right panel, the abstract graph with node embeddings. Suggested caption below.

\caption{Recasting audio as a graph. A log-mel spectrogram (left) is divided into patches that become graph nodes (middle); edges connect each node to its k nearest neighbours in feature space, linking related time--frequency regions irrespective of their grid distance. Graph neural networks then update node embeddings by message passing over this structure (right).}

Chapter 2 - Background (formal precision)

The report asks for a more formal Chapter 2: explicit notation, definitions before use, and an over-smoothing / graph-transformer discussion. The following drop-in blocks address each point; place them at the start of Section 2.2/2.3 and inside Section 2.3.7.

2a. Notation and terminology paragraph (start of Section 2.2)

\paragraph{Notation.} Scalars are written in lowercase italics (x), vectors in bold lowercase ($\mathbf{x}$), matrices in bold uppercase ($\mathbf{X}$), and sets in calligraphic type ($\mathcal{X}$). A dataset is $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ with inputs $\mathbf{x}_i \in \mathbb{R}^d$ and targets $\mathbf{y}_i$. A model f_{θ} with parameters θ produces predictions $\hat{\mathbf{y}}_i = f_{\theta}(\mathbf{x}_i)$. Throughout this thesis, the \emph{loss function} $\ell(\hat{\mathbf{y}}, \mathbf{y})$ measures the discrepancy on a single example, while the \emph{cost function} $J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$ is its average over the training set; training minimises $J(\theta)$ by stochastic gradient descent. For a network of L layers, $\mathbf{h}^{(l)}$ denotes the activations of \emph{hidden layer} $l \in \{1, \dots, L-1\}$ and the final layer L is the \emph{output layer}, whose activation (sigmoid, softmax, or identity) is chosen to match the target type.

2b. CNN computation (formal definition for Section 2.3.2)

Formally, a convolutional layer maps an input feature map $\mathbf{X} \in \mathbb{R}^{C_{in}} \times H \times W$ to $\mathbf{Y} \in \mathbb{R}^{C_{out}} \times H' \times W'$ via

$$\begin{equation} \mathbf{Y}[c, i, j] = b_c + \sum_{c'=1}^{C_{in}} \sum_{u=1}^{k_h} \sum_{v=1}^{k_w} \mathbf{K}_c[c', u, v] \mathbf{X}[c', s_h i + u - 1, s_w j + v - 1], \end{equation}$$

where $\mathbf{K}_c$ is the c -th learned kernel of size $k_h \times k_w$, b_c its bias and (s_h, s_w) the stride. The same kernel is applied at every spatial position (weight sharing), yielding shift-equivariant features with a number of parameters independent of the input size. In the audio context, H and W index time frames and frequency bands of a (mel-)spectrogram.

2c. RNN computation (formal definition for Section 2.3.3)

A recurrent layer processes a sequence $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ by maintaining a hidden state $\mathbf{h}_t \in \mathbb{R}^{N_h}$ updated as $\mathbf{h}_t = \phi(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b})$, where $\mathbf{W}_x \in \mathbb{R}^{N_h \times N_x}$ and $\mathbf{W}_h \in \mathbb{R}^{N_h \times N_h}$ are the input and recurrent weight matrices, shared across all time steps, and ϕ is a nonlinearity. The LSTM \citep{hochreiter1997long} augments this with a cell state $\mathbf{c}_t$ and input/forget/output gates $(\mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t)$ that control what is written to, retained in, and read from the cell:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

mitigating vanishing gradients over long sequences.

2d. General Transformer architecture (addition to Section 2.3.x, before AST)

The Transformer \cite{vaswani2017attention} dispenses with recurrence and instead relates all positions of a sequence directly through self-attention. Given token embeddings $\mathbf{X} \in \mathbb{R}^{N \times d}$, each attention head computes queries, keys and values $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ and outputs

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}}\right)\mathbf{V},$$

a data-dependent weighted average in which every token attends to every other token. A Transformer block combines multi-head attention with a position-wise feed-forward network, residual connections and layer normalisation; positional encodings inject order information that attention alone does not provide. Because attention weights are computed from the data rather than fixed by locality, Transformers capture long-range dependencies at the cost of $\mathcal{O}(N^2)$ complexity in the sequence length, and they lack the built-in locality bias of CNNs, which is one reason attention-based audio models benefit strongly from large-scale pretraining.

2e. GNN tasks and readout (addition to Section 2.3.7)

GNN prediction tasks are commonly grouped by the granularity of the target \cite{zhou2020graph}: \emph{node-level} tasks predict a label per node (e.g. classifying each time-frequency patch); \emph{edge-level} tasks predict the presence or weight of links between node pairs (e.g. affinity between two embeddings); and \emph{graph-level} tasks predict a property of the whole graph (e.g. the tag set of a clip). Graph-level prediction requires a \emph{readout} (aggregation) function that pools node embeddings into a fixed-size graph representation, $\mathbf{h}_G = \text{READOUT}(\{\mathbf{h}_v : v \in \mathcal{V}\})$, typically mean, max or sum pooling, chosen to be permutation-invariant so that the output does not depend on node ordering. The tagging models of Chapter~4 are graph-level predictors with average-pooling readout; the similarity and fingerprinting models of Chapter~5 produce clip-level embeddings by the same mechanism.

2f. Over-smoothing and low-pass filtering (addition to Section 2.3.7)

\paragraph{Over-smoothing.} Stacking many message-passing layers degrades GNN performance: each aggregation step averages a node's embedding with its neighbours', so with depth the embeddings of connected nodes converge and become indistinguishable \cite{li2018deeper, oono2020graph}. From a spectral perspective, the normalised aggregation operator acts as a low-pass filter on the graph signal: repeated application suppresses the high-frequency components that discriminate between nodes, leaving predominantly the smooth (low-frequency) components \cite{nt2019revisiting}. Intuitively, after L layers a node's receptive field covers its L -hop neighbourhood; when this neighbourhood covers most of the graph, all nodes aggregate nearly the same information. Common mitigations, used in this thesis, include restricting neighbourhood size via sparse k -NN graphs, dilated neighbour selection (Chapter~4), residual/initial connections, and keeping the number of message-passing layers per stage small.

2g. Graph Transformers (new short subsection after GNNs)

\paragraph{Graph Transformers.} Standard Transformers can be viewed as GNNs operating on a fully connected graph in which attention learns the edge weights. Graph Transformers \cite{dwivedi2021generalization, ying2021transformers} make this connection explicit: attention is computed over graph neighbourhoods or biased by structural encodings (shortest-path distances, Laplacian eigenvectors), combining the relational inductive bias of GNNs with the global receptive field of attention. They avoid over-smoothing by design, since attention weights are content-dependent rather than uniform averages, but inherit the quadratic cost of dense attention. The models in this thesis instead retain sparse k -NN message passing for efficiency, while adopting Transformer components (feed-forward blocks, residual connections,

positional encodings) within each graph block; extending them to full graph Transformers is discussed as future work in Chapter-6.

Chapter 3 - Hypernetworks for SED

3a. Multi-seed URBAN-SED results (update to Table 3.2 and discussion)

The amendment experiments retrained all models on URBAN-SED with 5 seeds (hypernet-sed repository, results/table2.md). LaTeX table and replacement discussion:

```
\begin{table}[t]\centering
\caption{Segment-wise F1 (\%) on the URBAN-SED test set: mean  $\pm$  standard
deviation over five random seeds. Each run is scored as the mean over its ten best
checkpoints by validation loss.}
\begin{tabular}{lcc}\toprule
Model & Overall F1 & Avg. class F1 \\ \midrule
Bi-baseline &  $62.74 \pm 0.19$  &  $62.27 \pm 0.30$  \\
Uni-baseline &  $61.19 \pm 0.45$  &  $60.87 \pm 0.58$  \\
HCRNN-32 &  $60.90 \pm 0.75$  &  $60.52 \pm 0.86$  \\
HCRNN-64 &  $61.06 \pm 0.45$  &  $60.68 \pm 0.65$  \\
HCRNN-128 &  $61.62 \pm 0.60$  &  $61.36 \pm 0.72$  \\
HCRNN-256 &  $61.72 \pm 0.69$  &  $61.36 \pm 0.85$  \\ \bottomrule
\end{tabular}\end{table}
```

Across five seeds, the unidirectional HCRNN variants improve upon the unidirectional baseline by a modest margin (HCRNN-128: $+0.43\%$ F1 on average), while the bidirectional baseline remains the strongest configuration overall. The standard deviations ($0.2\text{--}0.8$ F1) show that single-run differences of under one point --- as reported in the original submission --- fall within seed-to-seed variability, and the conclusions are accordingly softened: dynamic weight generation is competitive with, but does not consistently surpass, a bidirectional recurrence of comparable output dimensionality on this dataset. The per-class breakdown shows the hypernetwork's largest gains on classes with long, stationary activity (air conditioner, engine idling), consistent with the interpretation that input-conditioned scaling helps most where the recurrent dynamics must adapt to slowly varying context.

Honesty note: in the multi-seed reproduction the HCRNNs do NOT beat the bi-baseline, unlike the single-run thesis numbers. The draft above follows the report's guidance ('emphasise trends and limitations rather than single-number claims'). The TUT-SED Synthetic 2016 multi-seed rerun is still outstanding (remaining item R2).

3b. Hidden-dimension sweep (new paragraph answering the over-parameterisation concern)

To examine sensitivity to the hypernetwork's capacity, the hidden size $N_{\hat{h}}$ was swept over $\{32, 64, 128, 256\}$ (Table above). Performance varies by only 0.8% F1 across an $8\times$ range of hypernetwork width, with a weak upward trend saturating beyond $N_{\hat{h}} = 128$. The corresponding parameter counts (2.47M, 2.59M, 2.84M, 3.44M, versus 2.34M for the unidirectional baseline) confirm that the hypernetwork adds only 6--47% parameters, and that the observed differences between HCRNN-64 and HCRNN-128 are not an artefact of model order: both lie within one standard deviation of each other.

3c. Parameter counts, runtime, and compute details (new experimental-setup text)

```
\begin{table}[t]\centering
\caption{Parameter counts and measured walltime (single NVIDIA L40S, PyTorch 2.5.1,
CUDA 12.4, float32, no JIT compilation, batch 64 of 2\,s chunks, 8 data-loader
workers). Train step = forward + backward + Adam update; inference under
\texttt{no\_grad}.}
\begin{tabular}{lcc}\toprule
Model & Params (M) & Train step (ms) & Inference (ms/clip) \\ \midrule
Uni-baseline & 2.34 & 65 & 0.31 \\ \end{tabular}
```

Bi-baseline	& 5.76	& 69	& 0.33	\\
HCRNN-32	& 2.47	& 483	& 1.48	\\
HCRNN-64	& 2.59	& 483	& 1.53	\\
HCRNN-128	& 2.84	& 506	& 1.53	\\
HCRNN-256	& 3.44	& 505	& 1.53	\\ \bottomrule

Although the HCRNNs use fewer parameters than the bidirectional baseline, they train roughly $5\times$ slower per step in the present implementation: the per-time-step weight generation precludes the fused cuDNN LSTM kernel and executes as an explicit loop over time. Parameter count therefore does not translate into runtime efficiency, and walltime is reported alongside parameters throughout. No just-in-time compilation (`\texttt{torch.compile}` or TorchScript) was used for any model; mixed precision was not used for these experiments.

3d. Clarification of Equations 3.14-3.17 (insert after Eq. 3.16)

In Equations 3.14--3.16 the tensor--vector product $\mathbf{W}_{xz} \cdot \mathbf{z}_x$ denotes contraction over the embedding dimension: for $\mathbf{W}_{xz} \in \mathbb{R}^{N_h \times N_x \times N_z}$ and $\mathbf{z}_x \in \mathbb{R}^{N_z}$, the result is the matrix $\sum_{k=1}^{N_z} \mathbf{W}_{xz}[:, :, k] \mathbf{z}_x[k] \in \mathbb{R}^{N_h \times N_x}$, i.e. the generated weight matrix is a linear combination of N_z learned basis matrices with input-dependent coefficients. Storing these bases requires $N_h N_x N_z$ parameters per gate, which is prohibitive; the factorised form of Eq. 3.17 replaces the full generation by an input-dependent row-wise rescaling $\mathbf{z}$ of a single shared matrix, reducing the additional parameters to $\mathcal{O}(N_h N_z)$ per gate while retaining per-time-step adaptivity.

3e. Time-varying filters and silent periods (insert into the motivation)

The motivation for input-conditioned recurrence is that the statistics of a soundscape change over time: event onsets, overlapping sources and silent periods each call for different temporal integration. A fixed-weight LSTM applies the same transition function at every frame, so it must encode all of these regimes in a single parameterisation. The hypernetwork instead acts as a time-varying filter generator: during silent or stationary passages the scaling vector $\mathbf{z}_t$ can attenuate state updates, effectively lengthening the memory, while at event onsets it can sharpen the update dynamics. This adaptivity is the property being tested in the experiments that follow.

3f. LSTM as temporal pooling; 1-second segments (clarifications)

In this architecture the LSTM does not act as a temporal pooling stage: the CNN front-end preserves the frame rate (pooling is applied along frequency only), and the LSTM outputs one hidden state per frame, from which frame-wise class probabilities are predicted. Temporal aggregation occurs only at evaluation time, when frame-level predictions are compared against reference annotations in fixed-length segments.

Segment-based F1 with 1-second segments is used following the standard `\texttt{sed\_eval}` protocol [\cite{mesaros2016metrics}](#), in which predictions and references are compared per one-second block. This granularity matches the DCASE evaluation convention, is tolerant to small onset/offset deviations that are perceptually insignificant, and enables direct comparison with published URBAN-SED and TUT-SED results using the same tolerance.

3g. Table 3.3: random baseline and updated few-shot results

`\begin{table}[t]\centering`
`\caption{DCASE few-shot bioacoustic detection, Validation Set: event-based`
`precision, recall and F-score (\%), harmonic mean over subsets; mean  $\pm$  95\%`
`confidence interval over three trials.}`

```

\begin{tabular}{lccc}\toprule
Model & Precision & Recall & F-score \\ \midrule
Random baseline & $0.80 \pm 0.02$ & $16.68 \pm 0.30$ & $1.52 \pm 0.03$ \\
Proto Network & $18.06 \pm 0.39$ & $21.78 \pm 7.30$ & $19.44 \pm 3.23$ \\
H-Proto-1 & $19.05 \pm 2.05$ & $24.46 \pm 6.12$ & $21.14 \pm 2.53$ \\
H-Proto-2 & $16.29 \pm 0.99$ & $23.24 \pm 3.44$ & $19.13 \pm 1.80$ \\
H-Proto-3 & $17.82 \pm 2.27$ & $25.56 \pm 1.80$ & $20.96 \pm 1.89$ \\
H-Proto-All & $18.62 \pm 1.84$ & $25.94 \pm 0.62$ & $21.66 \pm 1.38$ \\
\bottomrule\end{tabular}\end{table}

```

A random-guessing baseline (uniformly sampled event predictions with matched duration statistics) achieves an F-score of only 1.5%, confirming that the absolute scores in this task reflect genuine difficulty rather than a high chance floor: few-shot bioacoustic detection requires generalising from five labelled events to unseen species and recording conditions, and even recent published systems on this benchmark remain below 60% F-score on the validation subsets. Direct comparison with state-of-the-art DCASE submissions is complicated by differing validation subsets across challenge releases and by ensembling and per-subset tuning in submitted systems; the comparison here therefore isolates the architectural contribution (hypernetwork gating) under a fixed pipeline.

3h. Hypernetworks placement (structural edit)

Editing instruction: move the general hypernetwork exposition (Ha et al. formulation, Eqs. 3.10-3.17 up to the $d(z)$ factorisation) into Chapter 2 as a new subsection 'Adaptive parameter generation: Hypernetworks' at the end of Section 2.3, and open Chapter 3 with the SED-specific instantiation. Bridging sentence for Chapter 3:

Chapter~2 (Section~2.3.x) introduced hypernetworks as a mechanism for generating the parameters of a main network conditioned on context. This chapter instantiates that mechanism for sound event detection: a recurrent hypernetwork generates per-time-step scaling of the LSTM weights of a CRNN (Section~3.2), and a graph-structured hypernetwork modulates a prototypical network for few-shot bioacoustic detection (Section~3.3).

Chapter 4 - ATGNN and LHGNN

4a. Rebuilt narrative before the acronyms (replacement opening for the ATGNN section)

The proposed model addresses audio tagging in three stages, each targeting a different kind of structure in the problem. First, a *patch graph* treats regions of the spectrogram as nodes and connects each region to the regions most similar to it in feature space, wherever they lie on the time--frequency grid; graph convolutions over this structure let harmonically or timbrally related regions exchange information directly (the Patch GNN, PGN). Second, a *patch--label graph* introduces one learnable embedding per class and connects each label embedding to the spectrogram regions that most resemble it, so that each class gathers evidence from its own supporting regions (the Patch--Label GNN, PLG). Third, a *label--label graph* lets label embeddings influence one another through a learned adjacency matrix, capturing the strong co-occurrence structure of audio tags (the Label--Label GNN, LLG). One PLG block followed by one LLG block is referred to as a Multi-Label GNN (MLG) block. The full architecture alternates PGN stages with MLG blocks and combines patch-level and label-level predictions at the output.

4b. Motivation for the label graphs, with measured evidence (new subsection text)

The label graphs are motivated by measurable structure in the data. An analysis of the official FSD50K annotations (51{,}197 clips, 200 classes) shows that 84.3% of clips carry more than one label (mean 2.99, median 3, up to 22 labels per clip), and that label co-occurrence is strong and highly structured: 169 of the 200 classes have at least one partner class whose conditional co-occurrence probability $P(j \mid i)$ exceeds \$0.5\$, in most cases approaching \$1.0\$. Part of this structure

is hierarchical, inherited from the AudioSet ontology (every `\emph{Electric guitar}` clip is also labelled `\emph{Guitar}` and `\emph{Music}`; `\emph{Music}` and `\emph{Musical instrument}` co-occur in 14{,}703 clips with $\approx 1\%$), and part reflects real-world regularities not imposed by the ontology (`\emph{Animal}` and `\emph{Wild animals}`, normalised PMI 0.80%). Co-occurrence is simultaneously sparse: most label pairs almost never co-occur. A learnable label--label adjacency is well-suited to this regime, since it can express a small number of strong, directed dependencies while leaving unrelated labels disconnected; and with ~ 3 labels per clip typically grounded in different regions of the spectrogram, per-label patch attachment allows each label to bind to its own evidence.

Figure available: `results/label_cooccurrence.png` in the ATGNN repository (labels-per-clip histogram, class frequency, $P(j|i)$ heatmap, per-class max P). Analysis script: `scripts/label_cooccurrence_fsd50k.py`.

4c. Prior work on label co-occurrence (related-work addition)

Exploiting label dependencies is well established in multi-label image classification: ML-GCN \citep{chen2019multi} propagates classifier weights over a co-occurrence graph derived from training statistics, and SSGL \citep{chen2019learning} attaches semantic label representations to image regions. In audio, label ontology structure has been used for AudioSet training \citep{jimenez2019sound}. ATGNN differs in learning the label adjacency end-to-end rather than fixing it from co-occurrence counts, and in coupling label embeddings to spectrogram patches at multiple scales of the encoder.

4d. Technical clarifications (kNN distance, flattening, nodes, positional encoding)

Each stage of the encoder produces a feature map $\mathbf{X}_t \in \mathbb{R}^{F \times T \times D}$, which is flattened along the spatial axes into a set of $N = F \cdot T$ node vectors $\mathbf{x}_i \in \mathbb{R}^D$; a node therefore corresponds to one spatial position of the CNN feature map, i.e. a receptive field over a local time--frequency region, not to a raw spectrogram pixel. The k -NN graph connects node i to the k nodes minimising the Euclidean distance $\|\mathbf{x}_i - \mathbf{x}_j\|_2$ in feature space; distances are computed on ℓ_2 -normalised features, making the criterion equivalent to cosine similarity. Because edges are recomputed from features at every block, the graph is dynamic and can connect distant grid positions. Since flattening discards grid coordinates, a learnable positional embedding is added to the node features after the stem, and the pyramid variant additionally biases the k -NN distance with a relative positional term; this restores (but does not enforce) sensitivity to time--frequency position, so the claim is that positional information is `\emph{available}` to the model rather than that the model is position-equivariant.

4e. ImageNet pretraining clarity (experimental-setup paragraph)

Pretraining is used as follows. The PGN encoder of ATGNN is initialised from the ImageNet-pretrained Pyramid ViG checkpoint of \citep{han2022vision}; the three-channel input stem is averaged to one channel, positional and relative positional embeddings are bicubically interpolated from the 224×224 image grid to the 128×1024 spectrogram grid, the ImageNet classification head is discarded, and all MLG components (label embeddings, PLG/LLG blocks, readout) are trained from scratch. LHGNN is reported both from scratch and with the analogous ImageNet initialisation of its ViG backbone. FSD50K- or AudioSet-based pretraining of the encoder was not used: FSD50K is the evaluation target (pretraining on it would leak supervision), and AudioSet pretraining is deliberately excluded so that the comparison isolates architecture from large-scale audio supervision; assessing audio-domain pretraining is left as future work.

4f. Table 4.3: uncertainty statement and reproduction runs

The FSD50K experiments in Table 4.3 report single training runs, as retraining every configuration across multiple seeds was computationally infeasible for the original submission. For the amended thesis, the released implementation was used

to retrain the key configurations end-to-end on 4 $\times$ NVIDIA L40S GPUs; single-seed reproduction gives eval-set mAP of 0.575 for the ATGNN-pyr-s encoder without MLG blocks, 0.580 for the full ATGNN-pyr-s, and 0.535 for LHGNN trained from scratch, consistent with the published values to within 0.01--0.04 mAP (the residual gap for LHGNN is attributable to the shorter 50-epoch schedule used in the reproduction). On ESC-50, where full multi-seed evaluation is affordable, results are reported as mean $\pm$ std over runs (Table 4.4), and the same convention is adopted for all rerun configurations. The encoder-only ablation additionally quantifies the MLG contribution on FSD50K: $+0.005$ mAP, mirroring the AudioSet ablation (0.335 vs 0.331).

If you complete the 3-seed reruns (remaining item R3), replace the single-seed sentence with mean $\pm$ std values.

4g. Parameters vs walltime (new table + efficiency discussion)

```
\begin{table}[t]\centering
\caption{Trainable parameters and measured walltime (single NVIDIA L40S, PyTorch
2.2.0, CUDA 11.8, float32, no JIT compilation; batch 24 clips of $1024 \times 128$
log-mel). Train step = forward + backward + Adam update.}
\begin{tabular}{lccc}\toprule
Model & Params (M) & Train step (ms) & Inference (ms/clip) \\ \midrule
ATGNN-pyr-s (encoder only) & 26.8 & 258 & 5.1 \\
ATGNN-pyr-s (full) & 35.4 & 290 & 5.7 \\
LHGNN & 31.1 & 643 & 14.5 \\ \bottomrule
\end{tabular}\end{table}
```

Parameter count is not a proxy for speed: LHGNN, with fewer parameters than the full ATGNN, is $\sim 2.2\times$ slower per training step and $\sim 2.5\times$ slower at inference in this measurement, because its per-block clustering and higher-order aggregation dominate runtime. Conversely, the MLG blocks add 32% more parameters but only $\sim 12\%$ walltime. Efficiency claims in this chapter are therefore stated in terms of both parameters and measured walltime. The parameter counts quoted here are trainable parameters; earlier counts that included frozen relative-position tables are corrected accordingly.

4h. Benchmark selection note (addition to the experiments section)

The baselines in Tables 4.1--4.4 (PANNS, PSLA, AST, ERANN and task baselines) are chosen because they share the training pipeline, features and evaluation protocol adopted here, allowing architecture-level comparison under matched conditions. More recent systems reach higher absolute mAP on these benchmarks, but do so through orthogonal factors --- larger-scale audio pretraining, self-supervised objectives, ensembling, or knowledge distillation --- that confound architectural comparison; the contribution of this chapter is the graph-based architecture, and the baselines are selected to isolate that contribution.

Chapter 5 - Music similarity and fingerprinting

5a. Locally Euclidean assumption (clarifying paragraph)

The k -NN graph construction assumes that the embedding space is *locally* Euclidean: within a small neighbourhood, straight-line (Euclidean) distance is a meaningful measure of perceptual similarity, even if the global geometry of the embedding manifold is curved. This is the standard assumption underlying manifold learning methods; it justifies using Euclidean distance only for selecting the k nearest neighbours (a local decision), while global relationships are mediated by message passing over the resulting graph rather than by global distances.

5b. Choice of GNN type (rationale paragraph)

A spatial graph convolution of the form $\mathbf{h}^{l+1} = \mathbf{h}^l \mathbf{A}^l \mathbf{W}^l$ is adopted rather than attention-based (GAT) or spectral alternatives for three reasons: (i) the affinities are already

learned by the relation network that constructs $\mathbf{A}$ (Eq. 5.3), so a second attention mechanism inside the convolution would duplicate that role; (ii) spectral convolutions assume a fixed graph, whereas here the graph is rebuilt per batch; and (iii) with small node sets per batch, the simple spatial form is the most parameter- and compute-efficient option, consistent with the design goal of a lightweight metric-learning head.

5c. Notation fixes around Eqs. 5.3-5.5 (editing instructions + text)

The symbol d is overloaded: $d(., .)$ is the distance in Eq. 5.3, d is also the embedding dimensionality, and $d(z)$ is the hypernetwork scaling in Chapter 3. Editing instruction: rename the distance to $d_E(., .)$ and state the convention. Matrices should be bold uppercase per the Chapter 2 notation paragraph (A , H , W). Insert after Eq. 5.3:

Here $d_E(\cdot, \cdot)$ denotes Euclidean distance between embeddings (denoted d_E to avoid conflict with the embedding dimensionality d), and the learned scales σ_i modulate the effective neighbourhood radius per node, allowing the network to sharpen or diffuse each node's affinities.

5d. ELU / GELU / cosine / Euclidean choices (rationale paragraph)

Activation and distance choices follow the role of each component. ELU is used in the graph convolution (Eq. 5.4) because its negative saturation keeps aggregated messages zero-centred, which stabilises repeated neighbourhood averaging; GELU is used inside feed-forward blocks for consistency with the Transformer-style blocks of Chapter 4. Euclidean distance is used where absolute distances between embeddings matter (graph construction, Eq. 5.3), while cosine similarity is used in the training objectives (Eqs. 5.5, 5.9), which compare directions of ℓ_2 -normalised embeddings and are invariant to their magnitude; this matches the retrieval setting, where ranking is performed by cosine similarity.

5e. A100 parallelism details (experimental-setup sentence)

Training used synchronous data parallelism (PyTorch DistributedDataParallel) across two NVIDIA A100 GPUs, with a per-GPU batch size of 128 and an effective batch size of 256; gradients are all-reduced at each step, and reported batch sizes refer to the effective (global) batch.

Verify against your original training logs: if `torch.nn.DataParallel` (single-process) was used instead of DDP, replace the first clause accordingly (remaining item R5).

5f. Near-identical retrievals claim (softened sentence)

Qualitative inspection of retrieval lists suggests that the model frequently ranks near-identical excerpts (same recording under different degradations) at the top; this is reported as an observed qualitative behaviour rather than a quantified property, as systematic retrieval-overlap analysis was not performed.

5g. Table 5.2 uncertainty (limitation statement, pending reruns)

The hit rates in Table 5.2 are computed from a single trained model per method over a fixed query set. Retraining all systems across seeds was not feasible within the amendment period; to characterise evaluation variability, the query sampling can be repeated over independent draws and the resulting variation reported alongside the point estimates. The comparison across noise and reverberation conditions, where GraFPrint's margin over the CNN- and Transformer-based baselines is largest at low SNR, is consistent across query lengths, which supports the robustness conclusion beyond any single operating point.

If you run the repeated-query-draw evaluation (remaining item R4), replace the second sentence with the measured mean $\pm$ CI per cell or per method.

5h. Metrics introduced linearly (editing instruction)

Editing instruction: before Section 5.2's results, add one sentence per metric in the order used (top-1 hit rate, precision@k, mean average precision), each defined before any table references it. Draft:

Retrieval quality is reported with three metrics. `\emph{Top-1 hit rate}` is the fraction of queries whose correct reference track is ranked first. `\emph{Precision@k}` is the fraction of the k highest-ranked items that are relevant. `\emph{Mean average precision}` (mAP) averages precision over the ranks of all relevant items, summarising the full ranking.

Chapter 6 - Conclusion

6a. Move the ATGNN explanation into Chapter 4 (editing instruction)

The report notes that the clearest explanation of ATGNN's contribution currently appears in the conclusion. Editing instruction: move that passage into Chapter 4 (either the chapter introduction or the opening of the ATGNN section, alongside draft 4a above), and keep Chapter 6 as synthesis. Replacement summary sentence for Chapter 6:

Chapter 4 demonstrated that treating spectrogram regions, class labels, and their interactions as graphs yields tagging models that are competitive with attention-based architectures at a fraction of the parameter count, and that the benefit of explicit label-graph modelling, while consistent, is secondary to the quality of the patch-level graph encoder.

Remaining items (not resolved in this document)

Experiments / code

- R1. Chapter 4: multi-seed FSD50K reruns for Table 4.3 uncertainty (3 seeds x 3 configurations, ~1 day on the current 4-GPU machine). Draft 4f already contains the fallback single-seed statement.
- R2. Chapter 3: TUT-SED Synthetic 2016 multi-seed rerun (dataset not currently on disk; URBAN-SED 5-seed study is complete).
- R3. Chapter 5: repeated-query-draw variability for Table 5.2 (requires the fingerprinting repo/indexes).
- R4. Chapter 5: verify DP vs DDP and per-GPU batch from original training logs (draft 5e assumes DDP).
- R5. Repository housekeeping: LICENSE, CITATION.cff, release tags (v1.0) and commit hashes for all repos; make ATGNN public or archive with DOI; clean/tag the two existing Chapter 5 repos.
- R6. Central reproducibility landing page (e.g. 'graph-neural-audio-analysis' repo with CODE_AVAILABILITY.md, DATASETS.md, per-chapter links).
- R7. Reproduction scripts named per table (reproduce_table_4_3.sh etc.) - thin wrappers around the existing run commands.
- R8. Chapter 1 audio-as-graph figure (composition suggested in 1d).

Writing / style pass (to do after substantive edits)

- G1. Choose one spelling convention (modelling/optimiser vs modeling/optimizer) - the current text mixes both (e.g. 'modeling paradigms' and 'modelling' in Chapter 1).
- G2. Standardise Figure vs Fig.; fix hyphenation (time-frequency vs time frequency; graph-based vs graph based is currently inconsistent).
- G3. Bibliography: consistent capitalisation (AudioSet, ImageNet, FSD50K in titles), complete venue metadata, deduplicate arXiv/published versions.
- G4. Typos flagged by examiners plus a full pass; e.g. 'use of of signal processing' in the opening paragraph of Chapter 1; 'attention mechanism that allow' -> 'attention mechanisms that allow'; missing full stops after research questions in Section 1.1.1 (items 2 and 3 end without '?').
- G5. Ensure every acronym is expanded at first use in each chapter (PGN/MLG/LLG are addressed by draft 4a; check SED, AT, MIR, VQT elsewhere).

- G6. Align equation notation with the Chapter 2 notation paragraph (bold matrices; d_E for distances; consistent loss vs cost terminology).

Response-to-corrections table

When assembling the response document for the examiners, map each item above to the corresponding page/section of the amended PDF. The drafts in this document are keyed (1a-6a) so they can be referenced directly from that table.